JAVA PROGRAMMING LABORATORY

INDUSTRY PROBLEM-SOLVING TASK

Java AWT, Applets, Event Handling and Layout Managers

Name of the Student	M.Chandana	**Register No.**	192472250
Programme / Section	CSE-AI	**Date**	22-08-2026
Course Code	CSA0925	**Course**	Programming in Java
Faculty	Dr Sivasankar Chandrasekaran	**Duration**	Lab Session

INDUSTRY TASK 1: Employee Performance & Payroll Dashboard

Problem Statement: A software company wants to develop a simple Employee Performance and Payroll Management System using Java AWT. The application should allow the HR team to enter employee details, process salary information, and generate a structured payroll report.

Functional Requirements

- Develop an AWT-based graphical user interface.
- Accept Employee ID, Employee Name, Department, Designation, Basic Salary, Working Days, and Performance Rating.
- Use suitable AWT components such as Label, TextField, Choice, Checkbox/CheckboxGroup, Button, TextArea, and List.
- Calculate Gross Salary, Performance Bonus, and Net Salary using suitable business rules defined by the student.
- Display the complete employee payroll report in a List or TextArea when Generate Report is clicked.
- Use GridBagLayout or BorderLayout for arranging the interface.
- Implement button events using ActionListener.
- Provide Submit/Generate Report and Reset functionality.
- Validate important inputs such as empty fields and numeric salary values.

Industry Context: The application simulates an HR payroll module used to capture employee information, perform basic payroll processing, and present a report for HR verification.

Expected Deliverables

- Java source code.
- Working GUI application with event handling.
- Sample input and output.
- Screenshot of the final application.
- Brief explanation of components, layout manager, and event handling used.

INDUSTRY TASK 2: Online Food Ordering and Billing System

Problem Statement: A restaurant wants to develop a simple Food Ordering and Billing Application for its counter staff. The application should allow customers to select food items, specify quantities, and generate a final bill.

Functional Requirements

- Develop the application using Java AWT.
- Create fields for Customer Name and Table Number.
- Provide Food Category, Food Item, and Quantity selection using appropriate AWT components.
- Provide Add Item, Generate Bill, Clear, Next, and Previous buttons.
- Display selected food items in a List or TextArea.
- Calculate item-wise cost, Subtotal, GST, and Final Bill Amount.
- Implement button click events using ActionListener.
- Use CardLayout to create separate screens for Customer/Order Details, Food Selection, and Bill/Order Summary.
- Validate quantity and ensure that an item is selected before adding it to the order.

Industry Context: The application represents a simplified restaurant POS (Point of Sale) system used for taking customer orders, processing prices, and generating a customer bill.

Expected Deliverables

- Java source code.
- Working multi-card AWT interface.
- Sample order and generated bill.
- Screenshot of each card/screen.
- Brief explanation of CardLayout, AWT components, and event handling.

Evaluation Rubric

Criterion	Weightage	Marks / Remarks
Problem Understanding	15%	
GUI Components and Layout	20%	
Event Handling	20%	
Functional Correctness	25%	
Input Validation and User Experience	10%	
Program Explanation / Viva	10%	

Student Instructions

1. The solution must be implemented in Java using the concepts covered in the laboratory syllabus.
2. Students should demonstrate compilation, execution, and output during the practical session.
3. Use meaningful variable names and appropriate comments.
4. Students must be able to explain the selected AWT components, layout manager, event handling mechanism, and processing logic.

Q1.

EmployeePayroll.java

```
import java.awt.*;
import java.awt.event.*;

public class EmployeePayroll extends Frame implements ActionListener {

    TextField id, name, salary, days;
    Choice dept, designation;
    CheckboxGroup rating;
    TextArea report;
    Button generate, reset;

    EmployeePayroll() {

        setTitle("Employee Payroll");
        setLayout(new GridLayout(9, 2, 5, 5));

        add(new Label("Employee ID"));
        id = new TextField();
        add(id);

        add(new Label("Employee Name"));
        name = new TextField();
        add(name);

        add(new Label("Department"));
        dept = new Choice();
        dept.add("IT");
        dept.add("HR");
        dept.add("Finance");
        add(dept);

        add(new Label("Designation"));
        designation = new Choice();
        designation.add("Developer");
        designation.add("Team Lead");
        designation.add("Manager");
        add(designation);

        add(new Label("Basic Salary"));
        salary = new TextField();
 add(salary);

        add(new Label("Working Days"));
```

```
        days = new TextField();
        add(days);

        add(new Label("Performance"));
        rating = new CheckboxGroup();

        Panel p = new Panel();
        p.add(new Checkbox("1", rating, false));
        p.add(new Checkbox("2", rating, false));
        p.add(new Checkbox("3", rating, false));
        p.add(new Checkbox("4", rating, false));
        p.add(new Checkbox("5", rating, false));
        add(p);

        generate = new Button("Generate Report");
        reset = new Button("Reset");

        add(generate);
        add(reset);

        report = new TextArea();
        add(report);

        generate.addActionListener(this);
        reset.addActionListener(this);

        setSize(500, 500);
setVisible(true);

        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });
    }

    public void actionPerformed(ActionEvent e) {

        if (e.getSource() == generate) {

            if (id.getText().isEmpty() ||
                name.getText().isEmpty() ||
                salary.getText().isEmpty()) {

                report.setText("Please fill all required fields.");
                return;
            }

            try {
```

```
        double basic = Double.parseDouble(salary.getText());

        double allowance = basic * 0.20;
        double gross = basic + allowance;

        Checkbox c = rating.getSelectedCheckbox();

        if (c == null) {
            report.setText("Select performance rating.");
            return;
        }

        int r = Integer.parseInt(c.getLabel());

        double bonus = 0;

        if (r == 5)
            bonus = basic * .20;
        else if (r == 4)
            bonus = basic * .15;
  else if (r == 3)
            bonus = basic * .10;
        else if (r == 2)
            bonus = basic * .05;

        double net = gross + bonus;

        report.setText(
            "EMPLOYEE PAYROLL REPORT\n\n" +
            "ID: " + id.getText() + "\n" +
            "Name: " + name.getText() + "\n" +
            "Department: " + dept.getSelectedItem() + "\n" +
            "Designation: " + designation.getSelectedItem() + "\n" +
            "Basic Salary: " + basic + "\n" +
            "Working Days: " + days.getText() + "\n" +
            "Rating: " + r + "\n\n" +
            "Gross Salary: " + gross + "\n" +
            "Performance Bonus: " + bonus + "\n" +
            "Net Salary: " + net
        );

    } catch (NumberFormatException ex) {

        report.setText("Salary must be numeric.");
    }
}

if (e.getSource() == reset) {

    id.setText("");
```

```
            name.setText("");
            salary.setText("");
            days.setText("");
            report.setText("");
            rating.setSelectedCheckbox(null);
        }
    }

    public static void main(String[] args) {
        new EmployeePayroll();
      }
    }
```

OUTPUT: